

We assessed commit `#6f6486e3a0df149690487316e20ca3d1e45f5cc7`

Notes for you/your team

Behavior

Misc notes:

- What does it do? (business purpose)
- Vulnerable Task Manager (VTM): a Django project/task manager with users, groups, projects, tasks, notes, file uploads, password reset, a REST API, and an OpenAI chatbot.
- Who does it do this for? (internal / external customer base)
- Security training/lab environment; default training account is `chris / test123`. Fixtures include `tm` and `outsider` style email addresses, suggesting co-mingled company/customer data.
- What kind of information will it hold?
- PII: DOB, SSN, email, names. Credentials: MD5 password hashes, 6-digit reset tokens, JWTs in cookies. Project/task/note content and uploaded files. Chat history.
- What are the different types of roles?
- Groups (RBAC): `admin_g`, `project_managers`, `team_member` via `taskManager/fixtures/auth_group_permissions.json / groups.json`.
- `is_superuser`: built-in Django full access (`admin / seth` in fixtures).
- `is_staff`: repurposed as a broad data-access bit in DRF viewsets (`taskManager/serializers.py`) and chatbot tools (`chatbot/tools.py`); not the same as superuser.
- Net effect: two overlapping authz paths to audit — group permissions plus `is_staff / is_superuser`.
- What aspects concern your client/customer/staff the most?
- Plaintext credential logging, weak password hashing, hardcoded `SECRET_KEY`, command/SQL/SSRF injection points, IDOR in `profile_by_id`, disabled CSRF, and broad AI tool data access.

Tech Stack

- Framework & Language
 - Django 5.1.4
 - Python 3
- 3rd party components, Examples:

Library	Functionality	Security Risk (1-5)	Why
<code>Django==5.1.4</code>	Web framework	5	Auth, sessions, routing, templates, ORM, middleware, CSRF. Misconfiguration is systemic.
<code>djangorestframework</code>	REST API	5	Exposes all model endpoints; serialization and permission boundaries.
<code>djangorestframework-simplejwt</code>	JWT auth	5	Custom JWT cookie auth; weak signing key and long lifetimes.
<code>redis</code>	Cache/rate limit	4	Failed-login counter; no auth config in settings.
<code>openai</code>	LLM client	4	Sends data to OpenRouter; API key protection and prompt injection matter.
<code>requests</code>	HTTP client	4	Fetches user-controlled URLs; SSRF/exfiltration risk.

Library	Functionality	Security Risk (1-5)	Why
<code>drf-spectacular</code>	OpenAPI docs	3	Public <code>/schema/</code> , <code>/swagger-ui/</code> , <code>/redoc/</code> expose API structure.
<code>django-filter</code>	API filtering	3	Enables search over all fields; can expose sensitive data.
<code>django-health-check</code>	Health endpoints	3	<code>/ht/</code> may leak operational info.
<code>python-decouple</code>	Configuration	3	Env var loader; not consistently used for secrets.
<code>psutil</code>	System metrics	2	Limited direct exposure.
<code>sqlparse</code>	SQL utilities	2	Internal use; limited direct impact.
<code>pytz</code>	Timezone support	1	Minimal security relevance.
<code>xlwt</code>	Excel export	1	Not observed in request paths.
<code>django-extensions</code>	Dev tools	1	Not part of runtime attack surface.
<code>pipenv</code>	Packaging	1	Not runtime.

- Datastore
 - SQLite (`vtmdb.sqlite3`) by default, with commented MySQL credentials.
 - Redis for failed-login counters.
 - Filesystem for uploaded media.

Brainstorming / Risks

- OpenAI API key usage and prompt injection; what data is sent and how is the response rendered?
- `xlwt` could lead to CSV/Excel injection if used for export with user content.
- MD5 password hashing is still in use; should identify how to pivot that.
- File upload/download paths use `os.system` and user-controlled names; path traversal and command execution possible.
- `is_staff` is used as a data bypass in both the DRF API and chatbot; ensure it does not overlap with `is_superuser` expectations.
- Text/title fields for projects/tasks/notes lack validation and flow straight into templates.
- No CSRF protection globally; several sensitive views are also `@csrf_exempt`.
- jQuery 1.8.3 / 1.10.2 and jQuery UI 1.9.2 are outdated.

Checklist of things to review

Risks

Authentication

- ☒ Identify login mechanisms
 - Web form login in `taskManager/views.py:login`.
 - Custom `JWTAuthenticationMiddleware` reads an `access_token` cookie and sets `request.user`.
 - DRF `TokenAuthentication` for `/api/` endpoints via `Authorization: Token ...`.
- ☐ Review credential storage
 - `MD5PasswordHasher` in `taskManager/settings.py:171`.
 - Plaintext `username:password` logged at `taskManager/views.py:106-110`.
- ☐ Review password reset flow
 - `forgot_password` uses raw SQL on `email` (`taskManager/views.py:888`).
 - 6-digit token generated from `os.urandom(6)` (`taskManager/views.py:893-898`).
 - `reset_password` does not require the old password (`taskManager/views.py:841-877`).

Authorization

- ☒ Identify Roles
 - Groups: `admin_g`, `project_managers`, `team_member` from `taskManager/fixtures/auth_group_permissions.json / groups.json`.
 - `is_superuser`: implicit full permissions in Django.
 - `is_staff`: custom broad bypass in DRF viewsets (`taskManager/serializers.py`) and chatbot tools (`chatbot/tools.py`).
- ☐ Identify sensitive/privileged endpoints
 - `taskManager:profile_by_id (/taskManager/profile/<user_id>)` - edit any user by ID.
 - `taskManager:manage_groups` and `taskManager:view_all_users` - admin functions.
 - `taskManager:project_create/edit/delete` - gated by `has_perm('taskManager.<action>_project')`.
 - `taskManager:upload, download, downloadprofilepic` - file read/write.
 - `taskManager:ping, taskManager:settings, taskManager:view_img` - no `@login_required`.
 - `chatbot:*` - AI tool read/write for projects/tasks/notes/users.
 - DRF `/api/*` viewsets - token auth + `is_staff` queryset bypass.
- ☐ Identify authz expectations specific to the business purpose of the app
 - Can non-privileged users view/alter other accounts? **Yes:** `profile_by_id` loads/edits any user by numeric ID, including DOB/SSN and password.
 - Can users escalate access? **Yes:** `profile_by_id` adds arbitrary groups from `request.POST['groups']` (mass assignment).
 - Separation of duties: `can_create/edit/delete_project` permissions are group-derived; verify no single group holds all three and that `manage_groups` cannot be chained.
- ☐ Identify authorization filters/functions
 - `@login_required`, `@user_passes_test(...)`, and `belongs_to_project(user, project_id)`.
 - `JWTAuthenticationMiddleware` overwrites `request.user` from a cookie before Django auth checks.
 - DRF uses separate `TokenAuthentication` + `IsAuthenticated / is_staff` scoping.
- Broken Access Control
 - ☐ Insecure Direct Object Reference
 - `profile_by_id / profile_view` (arbitrary `user_id`).
 - `download / downloadprofilepic` (arbitrary `file_id / user_id`).
 - `task_details/edit/delete/complete` and `note_*` check project membership partially or not at all (# `TODO` comments in `task_delete` and `task_complete`).
 - ☐ Missing Function Level Access Control
 - `task_delete` and `task_complete` have explicit # `TODO` comments about missing authorization.
 - `ping, tm_settings, view_img` have no `@login_required`.
 - ☐ Verify Authorization Filters
 - Confirm `is_staff` gated `get_queryset()` in DRF and chatbot `_project_accessible/_task_accessible/_note_accessible` do not leak data to unintended staff.
- Generic authz flaws
 - ☐ Sensitive Data Exposure
 - DOB/SSN readable/writable via `profile_by_id`; `tm_settings` renders `request.META`; chatbot `search_database/get_users` expose broad user/profile/note content.
 - ☐ Mass Assignment
 - `profile_by_id` accepts `groups` from `POST` and adds the target user to any named group.
 - ☐ Business Logic Flaws
 - `forgot_password` SQLi; Redis lockout is username-only with no CAPTCHA.
 - ☐ Are CSRF protections applied correctly
 - CSRF middleware is globally disabled and several state-changing views carry `@csrf_exempt`.
 - ☐ Are users forced to re-assert credentials for critical side effects?
 - `change_password` has no current-password check.
 - `profile_by_id` can set a different user's password with no re-auth.

Auditing/Logging

- ☐ Logging config in `taskManager/settings.py` captures plaintext passwords at `DEBUG` level to `mysite.log`.

Injection

- ❑ Raw SQL via string interpolation in `taskManager/views.py:search`, `forgot_password`, and `project_details`.
- ❑ Command execution via `subprocess.getoutput` in `taskManager/views.py:ping` with a weak regex blacklist.
- ❑ `os.system` shell move in `taskManager/misc.py:store_uploaded_file` and `store_uploaded_img`.
- ❑ `requests.get(url)` in `taskManager/views.py:upload` for user-supplied URLs (SSRF).

Cryptography

- ❑ `MD5PasswordHasher` for password storage.
- ❑ Hardcoded `SECRET_KEY = 'secret'` reused as `SIMPLE_JWT['SIGNING_KEY']`.
- ❑ 365-day access/refresh tokens with `ROTATE_REFRESH_TOKENS` and `BLACKLIST_AFTER_ROTATION` disabled.

Configuration

- ❑ `DEBUG = True` and `ALLOWED_HOSTS = ['*']`.
- ❑ No `SESSION_COOKIE_*`, `CSRF_COOKIE_*`, or `SECURE_*` hardening.
- ❑ OpenAI settings default to OpenRouter without key validation.

Mapping / Routes

Full route map can be generated with `python manage.py show_urls`. Key routes are listed below.

URL	Name	View
/	index	taskManager.views.index
/taskManager/	index	taskManager.views.index
/taskManager/dashboard/	dashboard	taskManager.views.dashboard
/taskManager/login/	login	taskManager.views.login
/taskManager/logout/	logout	taskManager.views.logout_view
/taskManager/register/	register	taskManager.views.register
/taskManager/profile/	profile	taskManager.views.profile
/taskManager/profile/<user_id>	profile_by_id	taskManager.views.profile_by_id
/taskManager/profile_view/<user_id>	profile_view	taskManager.views.profile_view
/taskManager/change_password/	change_password	taskManager.views.change_password
/taskManager/forgot_password/	forgot_password	taskManager.views.forgot_password
/taskManager/reset_password/	reset_password	taskManager.views.reset_password
/taskManager/manage_groups/	manage_groups	taskManager.views.manage_groups
/taskManager/view_all_users/	view_all_users	taskManager.views.view_all_users
/taskManager/project_create/	project_create	taskManager.views.project_create
/taskManager/project_list/	project_list	taskManager.views.project_list
/taskManager/project_delete/<project_id>/	project_delete	taskManager.views.project_delete
/taskManager/project_details/<project_id>/	project_details	taskManager.views.project_details
/taskManager/<project_id>/edit_project/	project_edit	taskManager.views.project_edit
/taskManager/<project_id>/manage_tasks/	manage_tasks	taskManager.views.manage_tasks
/taskManager/<project_id>/upload/	upload	taskManager.views.upload
/taskManager/<project_id>/<task_id>/	task_details	taskManager.views.task_details
/taskManager/<project_id>/<task_id>/note_create/	note_create	taskManager.views.note_create
/taskManager/<project_id>/<task_id>/note_edit/<note_id>	note_edit	taskManager.views.note_edit

URL	Name	View
/taskManager/<project_id>/<task_id>/note_delete/<note_id>	note_delete	taskManager.views.note_delete
/taskManager/<project_id>/task_create/	task_create	taskManager.views.task_create
/taskManager/<project_id>/task_edit/<task_id>	task_edit	taskManager.views.task_edit
/taskManager/<project_id>/task_delete/<task_id>	task_delete	taskManager.views.task_delete
/taskManager/<project_id>/task_complete/<task_id>	task_complete	taskManager.views.task_complete
/taskManager/task_list/	task_list	taskManager.views.task_list
/taskManager/search/	search	taskManager.views.search
/taskManager/download/<file_id>/	download	taskManager.views.download
/taskManager/downloadprofilepic/<user_id>/	download_profile_pic	taskManager.views.download_profile_pic
/taskManager/view_img/	view_img	taskManager.views.view_img
/taskManager/settings/	settings	taskManager.views.tm_settings
/taskManager/ping/	ping	taskManager.views.ping
/api/users/	user-list	taskManager.serializers.UserViewSet
/api/userprofiles/	userprofile-list	taskManager.serializers.UserProfileViewSet
/api/projects/	project-list	taskManager.serializers.ProjectViewSet
/api/tasks/	task-list	taskManager.serializers.TaskViewSet
/api/notes/	notes-list	taskManager.serializers.NotesViewSet
/api/files/	file-list	taskManager.serializers.FileViewSet
/api-token/	-	rest_framework.authtoken.views.view
/api-auth/login/	-	django.contrib.auth.views.view
/api-auth/logout/	-	django.contrib.auth.views.view
/schema/	schema	drf_spectacular.views.view
/swagger-ui/	swagger-ui	drf_spectacular.views.view
/redoc/	redoc	drf_spectacular.views.view
/chat/page/	chat_page	chatbot.views.chat_page
/chat/send/	chat_send	chatbot.views.chat_send
/chat/stream/	chat_stream	chatbot.views.chat_stream
/chat/sessions/	session_list	chatbot.views.session_list
/chat/session/new/	session_new	chatbot.views.session_new
/chat/session//messages/	session_messages	chatbot.views.session_messages
/chat/session//delete/	session_delete	chatbot.views.session_delete
/ht/	health_check	health_check.views.view
/admin/	-	django.contrib.admin.sites.index
/static/	-	django.views.static.serve
/uploads/	-	django.views.static.serve

High

- ☐ taskManager:profile_by_id - edit any user by ID, mass-assign groups, set password

- ☐ taskManager:ping - unauthenticated command execution with weak blacklist
- ☐ taskManager:forgot_password - SQL injection via raw email query
- ☐ taskManager:upload - SSRF and arbitrary file fetch
- ☐ taskManager:settings - unauthenticated `request.META` dump
- ☐ taskManager:view_img - unauthenticated arbitrary URL rendering
- ☐ taskManager:download / downloadprofilepic - IDOR file/profile image access
- ☐ chatbot:send / chatbot:stream - AI tool access to broad data and write operations
- ☐ /api/userprofiles/ - exposes DOB/SSN/reset_token to authenticated users
- ☐ taskManager:change_password - no current-password or CSRF check

Mapping / Authorization Decorators

- ☐ @login_required - used on most taskManager and chatbot views.
- ☐ @user_passes_test(can_create_project) - taskManager/views.py:project_create.
- ☐ @user_passes_test(can_edit_project) - taskManager/views.py:project_edit.
- ☐ @user_passes_test(can_delete_project) - taskManager/views.py:project_delete.
- ☐ @user_passes_test(lambda u: u.is_superuser) - taskManager/views.py:view_all_users.
- ☐ @csrf_exempt - profile_by_id, reset_password, forgot_password, change_password, ping; CSRF middleware is also disabled.
- ☐ @require_POST - chatbot/views.py:chat_send, session_new, session_delete.

Mapping / Files

- ☐ taskManager/settings.py
 - ☐ Hardcoded SECRET_KEY = 'secret', also used as JWT SIGNING_KEY.
 - ☐ DEBUG = True, ALLOWED_HOSTS = ['*'].
 - ☐ SIMPLE_JWT 365-day lifetimes, rotation and blacklisting disabled.
 - ☐ PASSWORD_HASHERS = MD5PasswordHasher.
 - ☐ TokenAuthentication + IsAuthenticated for DRF.
 - ☐ No secure cookie / HSTS / CSP settings.
- ☐ taskManager/middleware.py
 - ☐ JWTAuthenticationMiddleware decodes access_token cookie and overwrites request.user.
- ☐ taskManager/urls.py
 - ☐ Registers /api/ DRF router, /admin/, /api-token/, /schema/, /swagger-ui/, /redoc/, chat/, ht/, and static/media.
- ☐ taskManager/serializers.py
 - ☐ DRF viewsets and is_staff scoping; UserProfileViewSet serializes DOB/SSN/reset_token.
- ☐ taskManager/forms.py
 - ☐ ProfileForm / ProjectFileForm; profile_by_id reads extra fields from POST directly.
- ☐ taskManager/views.py
 - ☐ Main view surface: login, upload, SQL/SSRF/command injection, password reset, IDOR.
- ☐ taskManager/misc.py
 - ☐ os.system shell moves for uploaded files.
- ☐ chatbot/tools.py
 - ☐ ReAct tools: get_users, search_database, add/update_* for projects/tasks/notes.
- ☐ chatbot/views.py
 - ☐ OpenAI streaming and chat session management.
- ☐ requirements.txt
 - ☐ Django/DRF/JWT/OpenAI/Redis; no django-csp or security headers package.